

MACHINE LEARNING BASICS



Instructors: Mohammadreza A. Dehaqani, Babak n. Arabi

Alireza Akhoundi, Parisa Yahyapour, Mehdi Moosavi, Hossein Noroozi

Fall 2025

Homework 5

Question 1: Maximum Likelihood Estimation

Let $x_1, x_2, \dots, x_n$ be independent samples from the following distribution:

$$P(x \mid \theta) = \theta x^{-\theta-1}, \quad \text{where } \theta > 1, x \geq 1.$$

Find the maximum likelihood estimator of θ .

Question 2: Logistic Regression with One Feature

We are given a sample in which each point has only one feature. Consider a binary classification problem in which sample values $x \in \mathbb{R}$ are drawn randomly from two different class distributions. The first class, with label $y = 0$, has its mean to the left of the mean of the second class, with label $y = 1$. We will use a modified version of logistic regression to classify these data points.

We model the posterior probability at a test point $z \in \mathbb{R}$ as

$$P(y = 1 \mid z) = s(z - \alpha),$$

where $\alpha \in \mathbb{R}$ is the sole parameter we are trying to learn and

$$s(\gamma) = \frac{1}{1 + e^{-\gamma}}$$

is the logistic function. The decision boundary is $z = \alpha$ (because $s(z) = \frac{1}{2}$ there).

We will learn the parameter α by performing gradient descent on the logistic loss function (a.k.a. cross-entropy). That is, for a data point x with label $y \in \{0, 1\}$, we find the α that minimizes

$$J(\alpha) = -y \ln s(x - \alpha) - (1 - y) \ln(1 - s(x - \alpha)).$$

Derive the stochastic gradient descent update for J with step size $\epsilon > 0$, given a sample value x and a label y . *Hint: feel free to use s as an abbreviation for $s(x - \alpha)$.*

Question 3: L2-Regularized Linear Regression with Newton's Method

Recall that the objective function for L2-regularized linear regression is

$$J(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2,$$

where X is the design matrix (the rows of X are the data points).

The global minimizer of J is given by

$$\mathbf{w}^* = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}.$$

Consider running Newton's method to minimize J . Let $\mathbf{w}_0$ be an arbitrary initial guess for Newton's method. Show that $\mathbf{w}_1$, the value of the weights after one Newton step, is equal to $\mathbf{w}^*$.

Question 4: Polynomial Regression and Ridge Regularization

In this question you will explore polynomial regression and ridge regularization. In the first part you will visualize polynomial fits of different degrees. In the second part you will study how both the polynomial degree and the ridge regularization parameter affect the training and validation errors on a dataset where overfitting is more pronounced.

Data pre-processing. You are given two CSV files:

- **data_A.csv:** a dataset with two columns x and y . Use this dataset as a single training set; you do not need to create a validation or validation set for this part.
- **data_B.csv:** a dataset with two columns x and y . Randomly shuffle the data and split it into two disjoint sets of equal size: a training set and a validation set.

For each chosen polynomial degree d , construct the design matrix by mapping each scalar input x to the feature vector

$$\phi_d(x) = [1, x, x^2, \dots, x^d]^\top,$$

and use the same mapping consistently for the training and validation sets where applicable. You will be given the instructions for the setting the value of d in the task description.

Task description. In both parts you will use polynomial regression models of the form

$$h_\theta(x) = \theta^\top \phi_d(x),$$

and train them by minimizing the squared error.

Part (A) – Dataset data_A.csv. Using the data in **data_A.csv**, fit polynomial regression models with degrees $d = 1, 3, 9$ by least squares, using the closed-form normal equation

$$\theta = (X^\top X)^{-1} X^\top y.$$

On a dense grid of x -values covering the range of the data, plot the training data points and the fitted curves for degrees $d = 1, 3, 9$ on the same figure.

Part (B) – Dataset data_B.csv.

1. **Effect of polynomial degree.** For **data_B.csv**, consider polynomial degrees $d = 1, 2, \dots, 12$. For each degree d : construct the design matrices for the training and validation sets, fit the polynomial regression model using the normal equation on the training set, and compute the mean squared error (MSE) on both the training and validation sets. Plot the training MSE and validation MSE as functions of the polynomial degree d on the same figure.
2. **Effect of ridge regularization.** Fix $d = 12$. For this fixed degree, fit ridge regression models on the training portion of **data_B.csv** for a range of regularization strengths λ using the closed-form solution

$$\theta_\lambda = (X^\top X + \lambda I)^{-1} X^\top y,$$

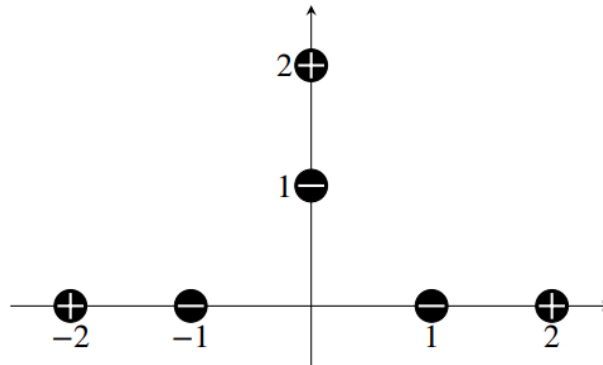
where X and y come from the training set. Use 50 values of λ spaced on a logarithmic scale between 10^{-6} and 10^2 , for example by using `np.logspace(-6, 2, 50)`. For each λ , compute the training and validation MSE. Plot the training MSE and validation MSE as functions of λ , using a logarithmic scale on the λ -axis.

Questions on the results.

- For **data_A.csv**, compare the fitted curves for degrees $d = 1, 3, 9$ to the observed data. Which degrees clearly underfit the data, and does degree 9 show any signs of unnecessary complexity relative to the visible pattern in the points?
- For **data_B.csv**, based on the MSE vs. degree plot, identify the range of polynomial degrees that underfit and the range that overfit. Which degree (or small range of degrees) appears to give the best trade-off between training error and validation error on this dataset?
- For **data_B.csv**, based on the MSE vs. λ plot for the fixed degree, describe how increasing λ affects the training and validation errors. For which range of λ does the validation MSE reach its minimum, and what happens for very small and very large values of λ ?
- Comparing your observations from both datasets, briefly discuss how the complexity of the fitted polynomial and the choice of regularization parameter influence overfitting, and how ridge regularization helps control the behavior of high-degree polynomial models.

Question 5: Perceptron

Consider a dataset with 6 points on a 2D coordinate grid, where each point belongs to one of two classes: negative class $[-1, 0], [1, 0], [0, 1]$ and positive class $[-2, 0], [2, 0], [0, 2]$.



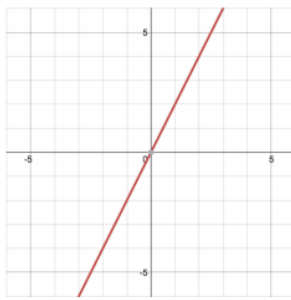
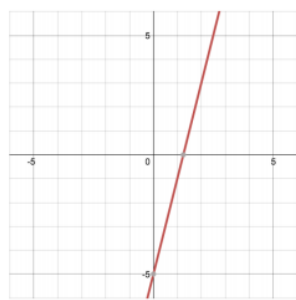
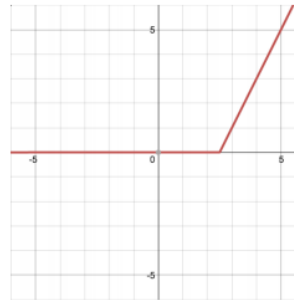
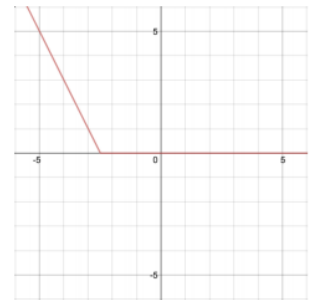
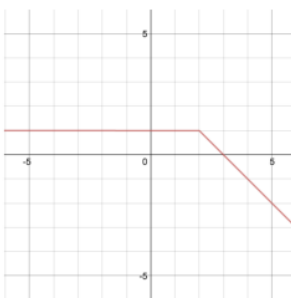
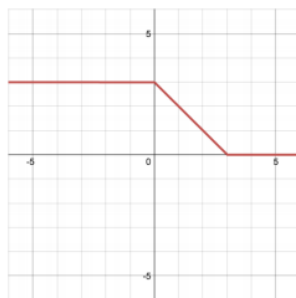
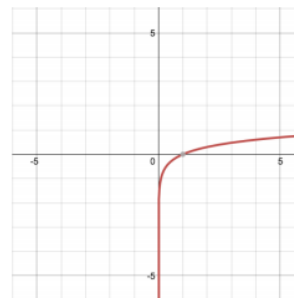
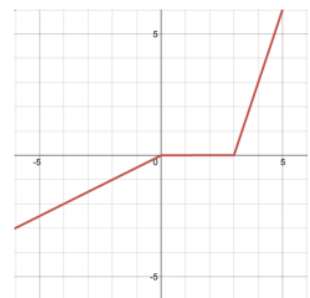
1. Suppose we run the perceptron algorithm with the initial weight vector set to $[0, 5]$. What is the updated weight vector after processing the data point $[0, 1]$?
2. How many iterations of the perceptron algorithm will run before the algorithm converges? Processing one data point counts as one iteration. If the algorithm never converges, write ∞ .

In the next few subparts, we'll consider transforming the data points by applying some modification to each of the data points. Then, we pass these modified data points into the perceptron algorithm.

For example, consider the transformation $[x, y] \rightarrow [x, y, x^2, 1]$. In this transformation, we add two extra dimensions: one whose value is always the square of the first coordinate, and one whose value is always the constant 1. For example, the point at $[2, 0]$ is transformed into a point at $[2, 0, 4, 1]$ in 4-dimensional space.

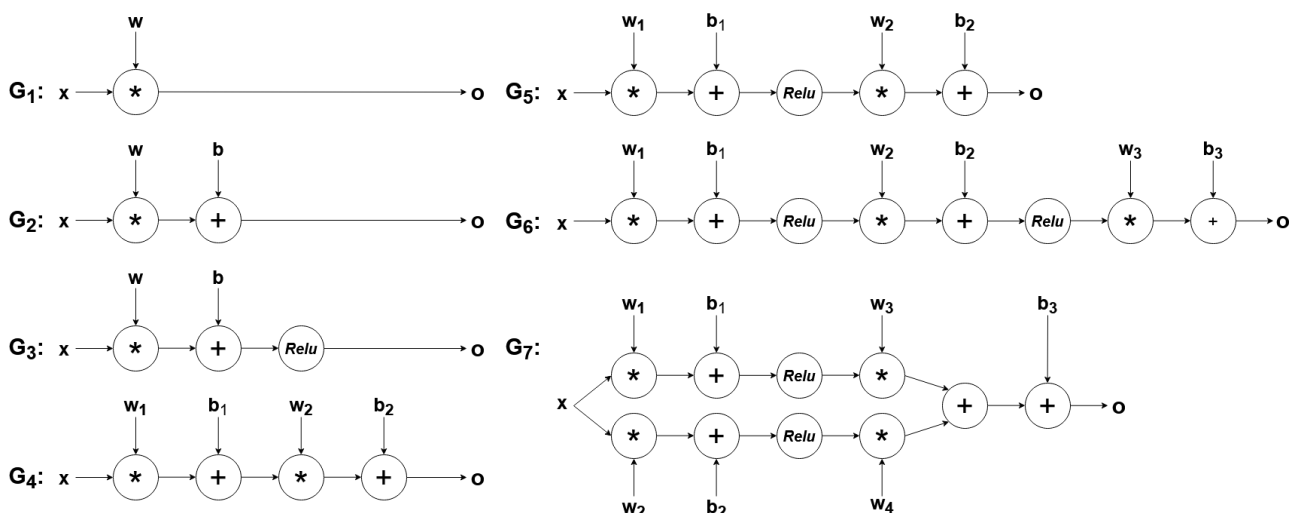
3. Which of the following data transformations will cause the perceptron algorithm to converge, when run on the transformed data? Select all that apply.
 - $[x, y] \rightarrow [y, x]$
 - $[x, y] \rightarrow [x, y, 1]$
 - $[x, y] \rightarrow [x, y, x^2, 1]$
 - $[x, y] \rightarrow [x, y, x^2 + y^2]$
 - None of the above.
4. Suppose we transform $[x, y]$ to $[x, y, x^2 + y^2, 1]$, and pass the transformed data points into the perceptron. Write one possible weight vector that the perceptron algorithm may converge to.
5. Construct another transformation (not equal to the ones above) that will allow the perceptron algorithm to converge.
 Hint: The transformation $[x, y] \rightarrow [x, y, x^2 + y^2, 1]$ allows the perceptron algorithm to converge.
 Fill in the blank: $[x, y] \rightarrow [x, y, _, 1]$.

Question 6: Neural Network Representations

(a) $2x$ (b) $4x - 5$ (c) $\begin{cases} 2x - 5 & x \geq 2.5 \\ 0 & x < 2.5 \end{cases}$ (d) $\begin{cases} -2x - 5 & x \leq -2.5 \\ 0 & x > -2.5 \end{cases}$ (e) $\begin{cases} -x + 3 & x \geq 2 \\ 1 & x < 2 \end{cases}$ (f) $\begin{cases} 3 & x \leq 0 \\ 3 - x & 0 < x \leq 3 \\ 0 & x > 3 \end{cases}$ (g) $\log(x)$ (h) $\begin{cases} 0.5x & x \leq 0 \\ 0 & 0 < x \leq 3 \\ 3x - 9 & x > 3 \end{cases}$

You are given a number of functions (a-h) of a single variable, x , which are graphed above. The computation graphs shown below will start off simple and get more complex, building up to neural networks.

- For each computation graph, indicate which of the functions above it is able to represent.
- For those that can, write out appropriate values of the weights and biases (for computation graphs G_1 to G_5).



Question 7: Loss function

In this question, we explore the connection between maximum likelihood and cross-entropy in neural network classification, considering extensions to noisy labels and alternative target representations.

1. Show that maximizing the likelihood for a multi-class neural network model in which the network outputs have the interpretation $y_k(\mathbf{x}, \mathbf{w}) = p(t_k = 1 \mid \mathbf{x})$ is equivalent to minimizing the cross-entropy error function $E(\mathbf{w}) = -\sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(\mathbf{x}_n, \mathbf{w})$ (where K is the number of mutually exclusive classes and t_{kn} is the k th component of the one-of- K target vector for data point n).
2. Consider a binary classification problem in which the target values are $t \in \{0, 1\}$, with a network output $y(x, w)$ that represents $p(t = 1 \mid x)$, and suppose that there is a probability ϵ that the class label on a training data point has been incorrectly set. Assuming i.i.d. data, write down the error function corresponding to the negative log likelihood. Verify that the usual cross-entropy error function is obtained when $\epsilon = 0$. Note that this error function makes the model robust to incorrectly labeled data, in contrast to the usual cross-entropy error function.
3. The cross-entropy error function for binary classification problems was derived for a network having a logistic-sigmoid output activation function, so that $0 \leq y(x, \mathbf{w}) \leq 1$, and data having target values $t \in \{0, 1\}$. Derive the corresponding error function if we consider a network having an output $-1 \leq y(x, \mathbf{w}) \leq 1$ and target values $t = 1$ for class C_1 and $t = -1$ for class C_2 . What would be the appropriate choice of output-unit activation function?

Question 8: Stochastic Stability and the “Dead ReLU”

Consider a single neuron $z = w^\top x + b$ with the rectified linear activation $a = \max(0, z)$. Let $x \in \mathbb{R}^n$ and $w \in \mathbb{R}^n$ be independent random vectors whose components are i.i.d., with $x \sim \mathcal{N}(0, \sigma_x^2 I)$ and $w \sim \mathcal{N}(0, \sigma_w^2 I)$. Assume that during backpropagation the ReLU derivative is taken as $\partial a / \partial z = \mathbf{1}_{z > 0}$ (i.e., the subgradient at $z = 0$ is 0).

1. **Distribution Proof.** Show that the pre-activation z follows a Gaussian distribution $\mathcal{N}(b, \sigma_z^2)$ and derive a formula for σ_z^2 in terms of n , σ_w^2 , and σ_x^2 .
2. **Activation Probability.** Using the standard normal cumulative distribution function Φ , derive an expression for the probability that the neuron is active, $P(a > 0)$, as a function of the bias b and the variance σ_z^2 .
3. **The 3σ Threshold.** For a bias initialized at $b = -3\sigma_z$, compute the numerical value of $P(a > 0)$. If a layer contains $N = 1000$ neurons initialized in this way, how many neurons are *expected* to be active?
4. **The Zero-Gradient Proof.** Show that if $z \leq 0$ for all samples in a training mini-batch, then the batch gradient with respect to the weights satisfies $\frac{\partial L}{\partial w} = 0$ and hence the weight update $\Delta w = 0$. Explain the mathematical implication of this result for “resurrecting” such a neuron during training.

Question 9: Variance Preservation and Xavier Initialization

Assume a linear neural network of depth L where each layer is defined as $y^{(l)} = W^{(l)} x^{(l)}$. The weights $W_{ij}^{(l)}$ are initialized i.i.d. with mean 0 and variance $\text{Var}(W)$.

1. **Symmetric Constraints:** Prove that to maintain constant signal variance in the forward pass, we require $\text{Var}(W) = 1/n_{\text{in}}$, and to maintain constant gradient variance in the backward pass, we require $\text{Var}(W) = 1/n_{\text{out}}$.
2. **The Glorot Synthesis:** Derive the Xavier variance $\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$ as a compromise between these two constraints.
3. **Depth-Induced Variance :** Let the layer gain be $G = n \cdot \text{Var}(W)$. Derive the expression for the output variance of the L -th layer, $\text{Var}(y^{(L)})$, as a function of the initial input variance $\text{Var}(x^{(0)})$.
4. **The Stability Bound :** Suppose the initialization contains a small error ϵ such that $n \cdot \text{Var}(W) = 1 + \epsilon$. Show that the output variance grows exponentially with depth L .
5. **Numerical Analysis:** Compute the amplification factor for a network where $L = 150$ and $\epsilon = 0.02$. Discuss why this exponential growth poses a significant risk for training in 16-bit floating-point (FP16) precision.

Question 10: Transfer Learning with Pre-trained Networks

In this problem, you will compare a shallow neural network trained from scratch with a deep pre-trained convolutional network on a small-scale image classification task. All experiments are conducted on the CIFAR-10 dataset (32×32 RGB images).

1. Data Pre-processing.

Construct a data pipeline that converts CIFAR-10 images into tensors and applies per-channel normalization using dataset-specific statistics.

2. Model Architectures.

Implement and train the following three models:

- *Model 1 (MLP Baseline)*: A Multi-Layer Perceptron operating on flattened CIFAR-10 images. The depth and hidden dimensionality of the MLP are left to your design choice.
- *Model 2 (Frozen VGG-11)*: A VGG-11 network pre-trained on ImageNet with all convolutional layers frozen. Replace the original classifier with a new classifier suitable for 10 classes.
- *Model 3 (Fine-Tuned VGG-11)*: The same architecture as Model 2, but with all layers trainable.

3. Training Procedure.

Train each model for 10 epochs using a mini-batch size of 512. Implement the training loop manually and explicitly perform the following steps for each mini-batch:

- (a) Zero the parameter gradients.
- (b) Compute the forward pass.
- (c) Compute the Cross-Entropy loss.
- (d) Perform backpropagation and update the model parameters.

4. Results.

For each model, plot:

- Training loss versus epoch
- Validation accuracy versus epoch

5. Discussion.

Based on the plots, briefly answer the following:

- **Convergence and Stability**: Contrast the "speed" and "smoothness" of the training curves between the MLP and the VGG models. Why does pre-trained initialization lead to a faster descent and more stable optimization than training from scratch?
- **Generalization Gap**: Despite the Frozen VGG having fixed convolutional weights, it significantly outperforms the MLP on the validation set. How does the *spatial inductive bias* of a CNN explain this performance gap relative to the flattened MLP?